

If a database does not need the extension and its related baggage anymore, you can issue a `DROP EXTENSION` statement and the extension will disappear from your database. Of course, if the extension has been installed as a database superuser, you need to issue the statement as a superuser too. With regards to the `orafce` example, as a superuser, you can do the following:

```
$ psql -U postgres forumdb
psql (16.0)
Type "help" for help.

forumdb=# DROP EXTENSION orafce;
DROP EXTENSION
```

As you can imagine, inspecting the extension list does not show the `orafce` entry anymore, and all the features, including the `DUAL` table, have disappeared:

```
forumdb=# \dx
           List of installed extensions
  Name      | Version | Schema | Description
-----+-----+-----+-----
pg_stat_statements | 1.10    | public | track planning and execution
statistics of all SQL stat
ements executed
plpgsql      | 1.0     | pg_catalog | PL/pgSQL procedural language
(2 rows)

forumdb=# SELECT * FROM oracle.dual;
ERROR:  relation "oracle.dual" does not exist
LINE 1: SELECT * FROM oracle.dual;
```

Having removed an extension from a single database does not remove it from other databases where you have executed an explicit `CREATE EXTENSION` function. It doesn't remove the extension files and libraries from the cluster share directory either.

The exact way of removing (un-deploying) the extension from your cluster depends on the way you first installed it in the cluster.

Removing an extension via pgxnclient

The `uninstall` command of `pgxn` performs the exact opposite action to the `install` command: it removes all files related to an extension. The command-line options are the same, and this leads us to execute a command as simple as the following one:

```
$ pgxn uninstall orafce --sudo --verbose
```

All extension-related files will be removed from the cluster's shared library directories. The extension is therefore gone forever, and if you need to install it again, you will need to restart from the very first step.

Removing a manually compiled extension

In order to remove an extension you manually compiled from sources, you need to use `make` again, this time with the `uninstall` command, in the directory where you extracted the downloaded compressed archive:

```
$ cd orafce-4.2.1
$ sudo make uninstall
```



Avoid mixing the management of extensions with different tools. If you installed an extension via `pgxnclient`, it is better to remove it with the same client; on the other hand, if you installed it from sources, use the same approach to remove it.

To summarize, you have seen how to deal with an extension with the PGXN client or manually by obtaining it through the PGXN infrastructure. In the following section, you will learn how to build your own extension.

Creating your own extension

In this section, we will build an extension from scratch so that you will better understand how they are made up. The idea is to let you know how to convert even your own SQL scripts into an extension, with all the advantages that an extension can provide in terms of manageability.

Defining an example extension

In order to demonstrate how to build your own extension, we are going to create a simple set of capabilities that apply to the forum database, providing some more features. In particular, we are going to define an extension named `tagext` that will provide a utility function that, given a particular tag within the `tag` table, will return the full path to that tag with all ancestors.

For example, the Linux tag is a child of the Operating Systems tag, and therefore the path to the Linux tag is Operating System > Linux.

In particular, we want our extension to provide us with a function named `tag_path` that, given a tag, provides the tag path as in the following example:

```
forumdb=> SELECT tag_path( 'Kubuntu' );
           tag_path
-----
Operating Systems > Linux > Ubuntu > Kubuntu
(1 row)
```

In the following sections, you will see how to reach the preceding result by implementing the example extension. All the required files are available in the book source code repository.

Creating extension files

Let's start with the control file first, where we insert some basic information about our extension. Create a file named `tagext.control` and place it into a folder, for example, `/src/tagext`. The file has the following content:

```
comment = 'Tag Programming Example Extension'
default_version = '1.0'
superuser      = false
relocatable    = true
```



In the Docker image used for the examples in this chapter, you will find the extension files in the `/src/tagext` folder and will be able to install the extension by entering into that folder.

The preceding control file contains a comment that describes the extension to other administrators, specifies the `default_version`, which is the version to be installed if none is specified by the user, and dictates that this extension can be installed by any user (`superuser = false`) and moved to any schema the user wishes to (`relocatable = true`).

Then comes the Makefile, that is, the file that will build and install the extension:

```
EXTENSION = tagext
DATA = tagext--1.0.sql

PG_CONFIG = pg_config
```

```
PGXS := $(shell $(PG_CONFIG) --pgxs)
include $(PGXS)
```

The Makefile is very simple and can be used as a skeleton for other extension Makefiles. In particular, we define the name of the extension we are going to manage via this Makefile, as well as the file to use for producing the extension content. This is specified in the `DATA` variable, and therefore we are instrumenting the system to use the `tagext--1.0.sql` file to create the objects this extension provides.

The trailing lines define the use of the PGXS build infrastructure and, in particular, are used to include the PGXS base Makefile, which is computed from the output of the `pg_config` command.

With all the infrastructure in place, it is now possible to define the content of the extension, therefore the `tagext--1.0.sql` file contains the definition of a function (see *Chapter 7, Server Side Programming*) that, given a specific tag, returns the text representation of the tag path with all the ancestors:

```
CREATE OR REPLACE FUNCTION tag_path( tag_to_search text )
RETURNS TEXT
AS $CODE$
DECLARE
    tag_path text;
    current_parent_pk int;
BEGIN

    tag_path = tag_to_search;

    SELECT parent
    INTO    current_parent_pk
    FROM    tags
    WHERE   tag = tag_to_search;

    -- here we must loop
    WHILE current_parent_pk IS NOT NULL LOOP
        SELECT parent, tag || ' > ' || tag_path
        INTO    current_parent_pk, tag_path
        FROM    tags
        WHERE   pk = current_parent_pk;
    END LOOP;
```

```

    RETURN tag_path;
END
$CODE$
LANGUAGE plpgsql;

```

The function works by taking a tag as an argument, then querying the `tags` table to get the parent primary key, and then looping on every parent tag. In every loop, the `tag_path` text string is enriched by the parent tag name so that the result is to have a string like `parent 1 > parent 2 > child`.

Once all the files are ready, we will have a situation like the following, with the Makefile, the control file, and the extension content file:

```

$ ls -l /src/tagext
Makefile
tagext--1.0.sql
tagext.control

```

Installing the extension

Having all the pieces in place, it is possible to use the Makefile to install (deploy) the extension in the cluster. Since the extension will be installed in the PostgreSQL directories, you could need to use a privileged user to install the extension:

```

$ sudo make install

```

And it is now possible to install the extension in the `forumdb` database using `CREATE EXTENSION` and then try to execute the function the extension defines:

```

forumdb=> CREATE EXTENSION tagext;
CREATE EXTENSION

                                List of installed extensions
   Name   | Version | Schema |
Description
-----+-----+-----+-----
-----
-----
-----

```

```

pg_stat_statements | 1.10      | public      | track planning and execution
statistics of all SQL stat
ements executed
plpgsql            | 1.0        | pg_catalog  | PL/pgSQL procedural language
tagext            | 1.0        | forum       | Tag Programming Example
Extension

```

```

forumdb=> SELECT tag_path( 'Kubuntu' );
           tag_path
-----
Operating Systems > Linux > Ubuntu > Kubuntu
(1 row)

```

The function works and can build up a tag tree or path for the specified tag, with all its ancestors, as well as PostgreSQL reporting that the extension is at version 1.0.

Creating an extension upgrade

Imagine we want to enrich our extension function so that the user can specify the tag separator in the path output. We can produce a new version of the function, drop the old one, and allow the user to upgrade the extension with the new content.

Let's start by creating an upgrade of the content of the extension, that is, the new function the extension provides. First of all, create a file named `tagext--1.0--1.1.sql` and place the following content in it:

```

DROP FUNCTION IF EXISTS tag_path( text );

CREATE OR REPLACE FUNCTION tag_path( tag_to_search text,
                                     delimiter text DEFAULT ' > ' )

RETURNS TEXT
AS $CODE$
DECLARE
    tag_path text;
    current_parent_pk int;
BEGIN

```

```

tag_path = tag_to_search;

SELECT parent
INTO    current_parent_pk
FROM    tags
WHERE   tag = tag_to_search;

-- here we must loop
WHILE current_parent_pk IS NOT NULL LOOP
    SELECT parent, tag || delimiter || tag_path
    INTO    current_parent_pk, tag_path
    FROM    tags
    WHERE   pk = current_parent_pk;
END LOOP;

RETURN tag_path;
END
$CODE$
LANGUAGE plpgsql;

```

The file first drops the older version of the function (if it exists and has been installed by the previous version of this extension). After that, a new function with an additional optional parameter is created. The function does exactly the same job as the previous one, but this time it exploits the variable `delimiter` to separate multiple tags.

Since we added a new file to the extension, we need to inform the Makefile about the file, and therefore we have to add the new file to the `DATA` variable so that the Makefile content looks like the following:

```

EXTENSION = tagext
DATA = tagext--1.0.sql tagext--1.0--1.1.sql

PG_CONFIG = pg_config
PGXS := $(shell $(PG_CONFIG) --pgxs)
include $(PGXS)

```

Performing an extension upgrade

With the new Makefile and the tagext--1.0--1.1.sql files, the situation on the disk looks like the following:

```
$ ls -l /src/tagext
Makefile
tagext--1.0--1.1.sql
tagext--1.0.sql
tagext.control
```

It is therefore now possible to install (deploy) the extension to the cluster, again running an install command:

```
$ make install
/bin/mkdir -p '/usr/local/share/postgresql/extension'
/bin/mkdir -p '/usr/local/share/postgresql/extension'
/usr/bin/install -c -m 644 ../tagext.control '/usr/local/share/postgresql/extension/'
/usr/bin/install -c -m 644 ../tagext--1.0.sql ../tagext--1.0--1.1.sql '/usr/local/share/postgresql/extension/'
```

And within the database, it is possible to upgrade the extension with ALTER EXTENSION:

```
forumdb=> ALTER EXTENSION tagext UPDATE TO '1.1';
ALTER EXTENSION

forumdb=> \dx tagext

               List of installed extensions
  Name  | Version | Schema | Description
-----+-----+-----+-----
 tagext | 1.1     | public | Tag Programming Example Extension
(1 row)
```

As you can see, the extension version is now 1.1, so it is possible to invoke the tag_path function with or without the new argument:

```
forumdb=> SELECT tag_path( 'Kubuntu' );
               tag_path
-----
```

```

Operating Systems > Linux > Ubuntu > Kubuntu
(1 row)

forumdb=> SELECT tag_path( 'Kubuntu', ' --> ' );
               tag_path
-----
Operating Systems --> Linux --> Ubuntu --> Kubuntu
(1 row)

```

You now know how to manage the whole life cycle of your extensions.

Summary

This chapter has introduced you to the extension ecosystem, a very rich and powerful system to package-related objects and manage them as a single unit. Extensions provide a way to add new features to your cluster and your databases and most notably provide a clear and concise way of building updates and repeatable installations, therefore easing the distribution of the features to other clusters and databases.

PostgreSQL ships with useful extensions provided within the `contrib` package; these extensions are developed directly by the PostgreSQL developers and therefore are very well integrated with the current PostgreSQL version. On the other hand, the PGXN network provides third-party extensions that can improve your cluster with new functionalities.

Thanks to the PGXS building infrastructure, creating an extension from scratch is comprehensive and quite easy, while thanks to tools such as `pgxnclient`, managing a lot of extensions can be automated.

In the next chapter, you will learn how to take care of the status and performance of your cluster, while *Chapter 19* will show you some other useful extensions.

Verify your knowledge

- What is an extension?

An extension is a collection of related database objects that can be installed, upgraded, or removed as a single unit. See the *Introducing extensions* section for more details.

- What is the `pgxnclient` command?

`pgxnclient` is a command that eases the usage of the PGXN by downloading, installing, and removing extensions. See the *Exploring the PGXN client* section for more details.

- What is an extension control file?

A control file is a text file that defines the main properties of an extension, like the name, the version, and the other dependencies. See the *Extension components* section for more details.

- How can you inspect which extensions have been created in a database?

The special `pg_extension` catalog provides information about installed extensions; in `psql`, the special `\dx` command shows a summary of installed extensions. See the *Viewing installed extensions* section for more details.

- How can you change the version of an extension (update)?

The `ALTER EXTENSION UPDATE TO` statement can be used to indicate to which version the extension has to be upgraded. See the *Altering an existing extension* section for more details.

References

- PostgreSQL official documentation about extensions: <https://www.postgresql.org/docs/current/extend-extensions.html>
- PostgreSQL official documentation about the extension build system (PGXS): <https://www.postgresql.org/docs/current/extend-pgxs.html>
- The `pgxnclient` official repository: <https://pypi.org/project/pgxnclient/>
- The `pgxnclient` official documentation: <https://pgxn.github.io/pgxnclient/>
- *PostgreSQL 11 Server Side Programming – Quick Start Guide*, Packt Publishing

Learn more on Discord

To join the Discord community for this book – where you can share feedback, ask questions to the author, and learn about new releases – follow the QR code below:

<https://discord.gg/jYWCjF6Tku>

